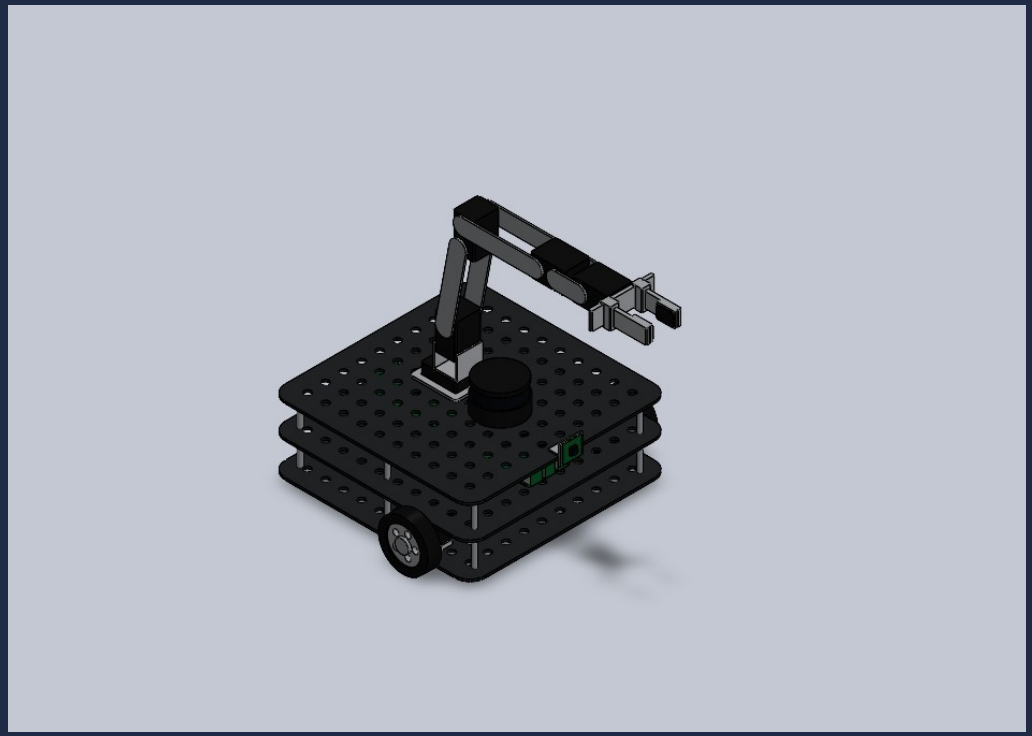
Motion Planning for a TurtleBot3 + OpenMANIPULATOR-X Mobile Manipulator

Comparison of RRT, RRT* and PRM



Subject: Robot Dynamic Control

Team: Kaviyashree T S, Barath K M, Jyothish S, Nabeeha Naaz H

Project review | 14 October 2026

Outline

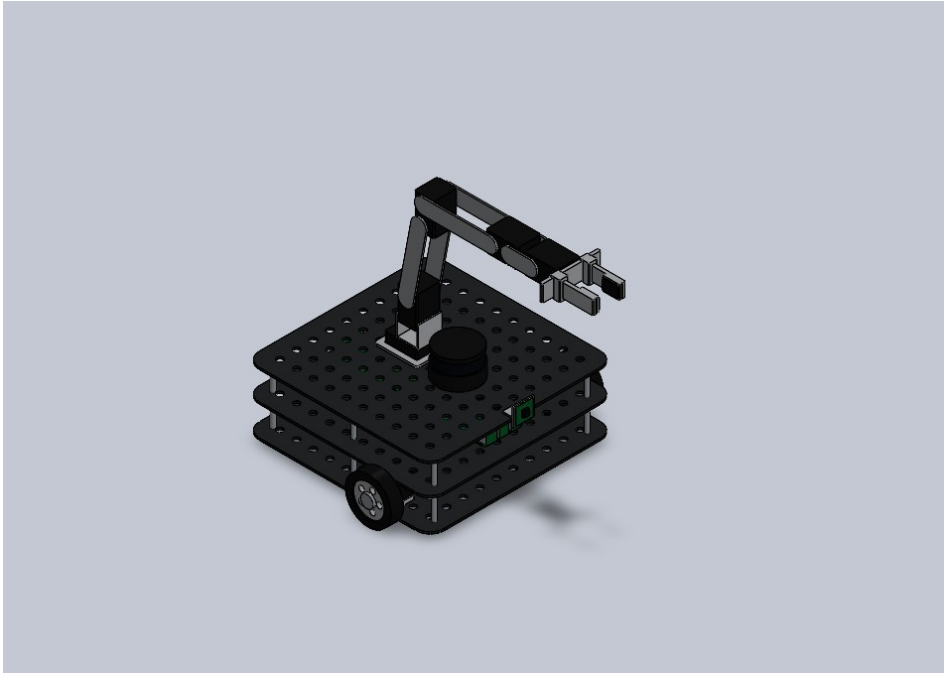
- 1 Abstract
- 2 Background and objectives
- 3 Literature review, research gap and novelty
- 4 Methodology: architecture, equations, algorithms
- 5 Results: simulation of each algorithm and justification
- 6 Conclusion and future work
- 7 References

1. Abstract

- Goal: study three motion-planning algorithms (RRT, RRT*, PRM) on a mobile manipulator
- Built a parametric SolidWorks 2025 model (19 parts, 2 assemblies, gripper) and a ROS 2 Humble package from one spec file
- Wrote and simulated each algorithm on the arm; scene where the straight path is blocked; 10 runs each; all succeeded
- RRT: fastest (43 ms) but longest paths (4.32 rad)
- RRT*: best paths (2.83 rad, 18 % above the 2.40 rad lower bound) with a 10 s budget
- PRM: 10.7 s to build the roadmap, 3.64 rad paths, reusable
- Next: full MoveIt 2 + OMPL + Gazebo study

2. Background

- Mobile manipulation: drive to a place, then work on objects there
- TurtleBot3 Waffle Pi: differential drive, LDS-01 LiDAR, Raspberry Pi 4, OpenCR, camera
- OpenMANIPULATOR-X: 4-DOF arm + gripper, DYNAMIXEL XM430-W210 servos
- The arm needs collision-free paths in joint space, and the search space grows with every joint
- Sampling-based planners in OMPL / MoveIt 2 are the usual answer



2. Problem statement and objectives

- Build a parametric SolidWorks 2025 model of the TurtleBot3 Waffle Pi with the OpenMANIPULATOR-X, including working revolute joints and a two-finger gripper.
- Export it as a STEP file and as a ROS 2 Humble description package (URDF, xacro, STL meshes, RViz launch).
- Check that the kinematics, joint limits and inertial values match the official ROBOTIS model.
- Explain RRT, RRT* and PRM mathematically, simulate each one on the robot and justify its behaviour.
- Compare the three on planning time, path length, success rate and node count.
- Get the package ready for the MoveIt 2 + OMPL + Gazebo comparison.

3. Literature review (1/3)

Title	Key findings	Limitations / Research gap
LaValle (1998) Rapidly-exploring random trees: a new tool for path planning [1]	Introduced RRT: a tree grown by sampling random states and extending the nearest node. Probabilistically complete, works in high-dimensional spaces, very quick to a first path.	Paths are jagged and not optimal; behaviour depends on step size and distance metric.
Kavraki et al. (1996) Probabilistic roadmaps for path planning in high-dimensional configuration spaces [2]	Introduced PRM: build a roadmap of free configurations once, then answer many queries by graph search. Good for static scenes and many goals.	Roadmap construction is costly; narrow passages are hard; roadmap must be rebuilt if obstacles move.
Kuffner & LaValle (2000) RRT-Connect: an efficient approach to single-query path planning [4]	Grows two trees (from start and goal) that try to connect greedily; much faster than basic RRT on manipulator-type problems.	Still not optimal, so paths need post-smoothing; only for problems where two states can be connected directly.
Karaman & Frazzoli (2011) Sampling-based algorithms for optimal motion planning [3]	Showed that RRT and PRM are not asymptotically optimal; proposed RRT* and PRM*, which converge to the optimal path with the same asymptotic complexity.	Convergence can be slow in higher dimensions; extra cost per sample from neighbour search and rewiring; radius needs tuning.

3. Literature review (2/3)

Title	Key findings	Limitations / Research gap
Gammell et al. (2014) Informed RRT* [13]	After the first solution, samples only inside an ellipsoid that can still improve it, so the path converges faster than RRT*.	Needs a first solution; advantage shrinks as dimension grows; assumes a simple distance-based cost.
Sucan et al. (2012) The Open Motion Planning Library [5]	Common C++ interface to many planners (RRT, RRT*, PRM and more) with benchmarking tools; it is the planning back-end of MoveIt.	Generic library: results depend on the robot model, collision checker and parameters chosen by the user.
Chitta et al. (2012) MoveIt! [6]	Brings kinematics, collision checking, planning and execution together for manipulators in ROS.	Needs per-robot configuration; arm-centred, so planning base and arm together takes extra set-up.

3. Literature review (3/3)

Title	Key findings	Limitations / Research gap
Macenski et al. (2022) Robot Operating System 2 [7]	Describes ROS 2 design: DDS communication, real-time support, multi-robot use and security.	General framework with no manipulation-specific planning comparison.
Koenig & Howard (2004) Design and use paradigms for Gazebo [8]	Open-source simulator with physics and sensor models, widely used to test robots before hardware.	Simulation-to-real gap; results are only as good as the URDF, inertia and friction values supplied.
ROBOTIS (e-Manual, GitHub) [11], [12] turtlebot3_manipulation	Official URDF, MoveIt configuration and Gazebo files for the TurtleBot3 + OpenMANIPULATOR-X.	No editable parametric CAD source and no planner comparison; meshes are fixed.

3. Research gap

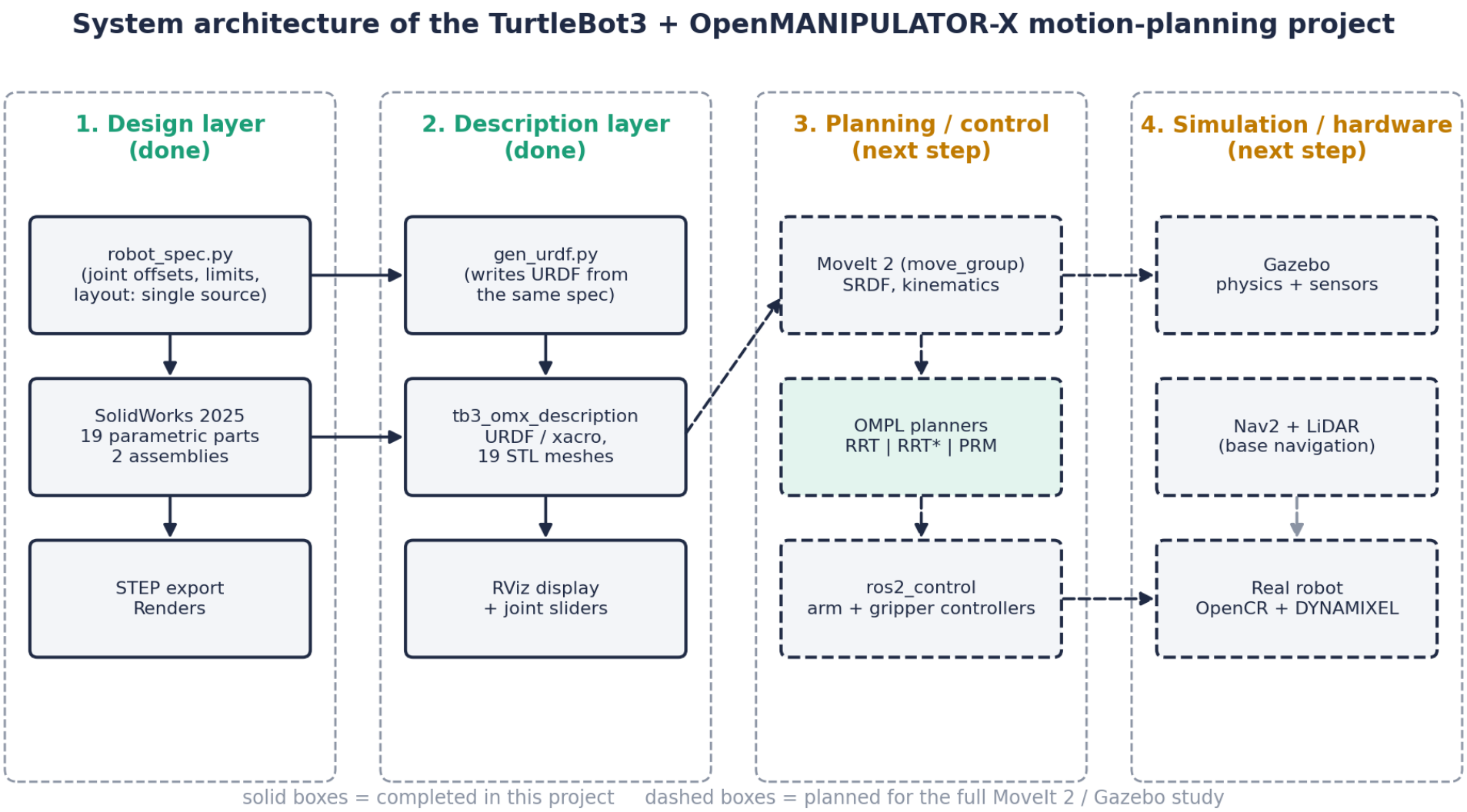
- Many planner studies use fixed-base industrial arms rather than a small ROS 2 mobile manipulator.
- Papers compare algorithms in general, but give no editable CAD-to-URDF model of the TurtleBot3 + OpenMANIPULATOR-X to repeat the study on.
- The official ROBOTIS package gives the model and a working MoveIt setup, but no side-by-side RRT / RRT* / PRM comparison and no parametric CAD.
- RRT, RRT* and PRM are often explained through theory and generic benchmarks; we also show each one running on the same robot with the same metrics.

3. Novelty of this work

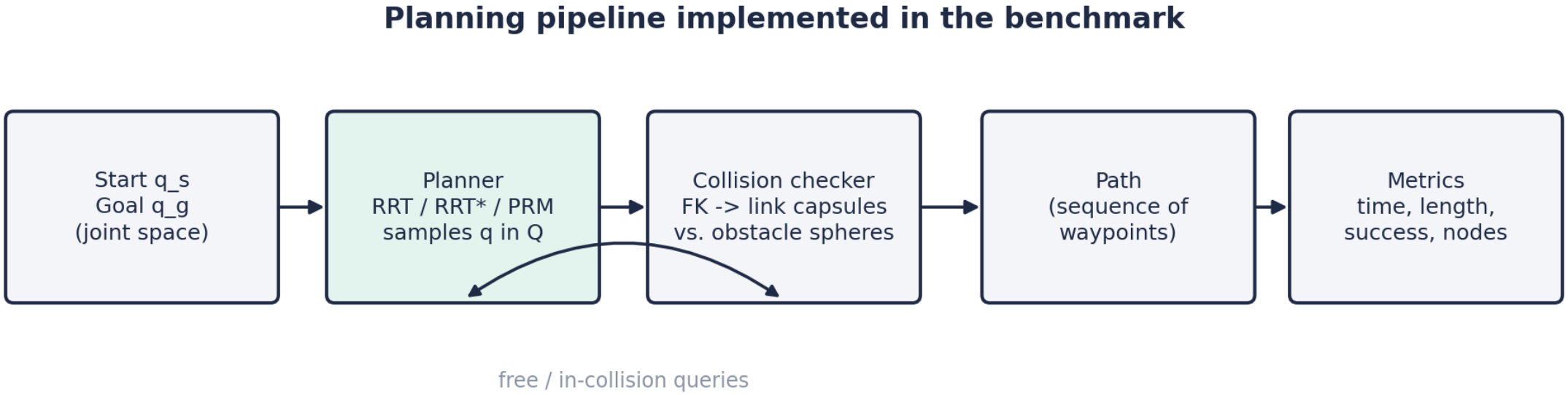
- One specification file (robot_spec.py) generates both the SolidWorks model and the URDF, so geometry and kinematics cannot drift apart. Changing one parameter, e.g. forearm length, updates the CAD and the simulation model together.
- A fully parametric SolidWorks 2025 model of the TurtleBot3 Waffle Pi + OpenMANIPULATOR-X (19 parts, working joint mates, two-finger gripper) with a drop-in ROS 2 package for MoveIt 2 and Gazebo.
- A like-for-like comparison of RRT, RRT* and PRM on this robot: same kinematics, same collision model, same start/goal and same metrics, with a simulation and a written justification for each algorithm.
- A reproducible benchmark script that the team can re-run, and a clear path to repeat the same study in MoveIt 2 / OMPL / Gazebo.

To the best of our knowledge, based on the sources reviewed.

4. System architecture



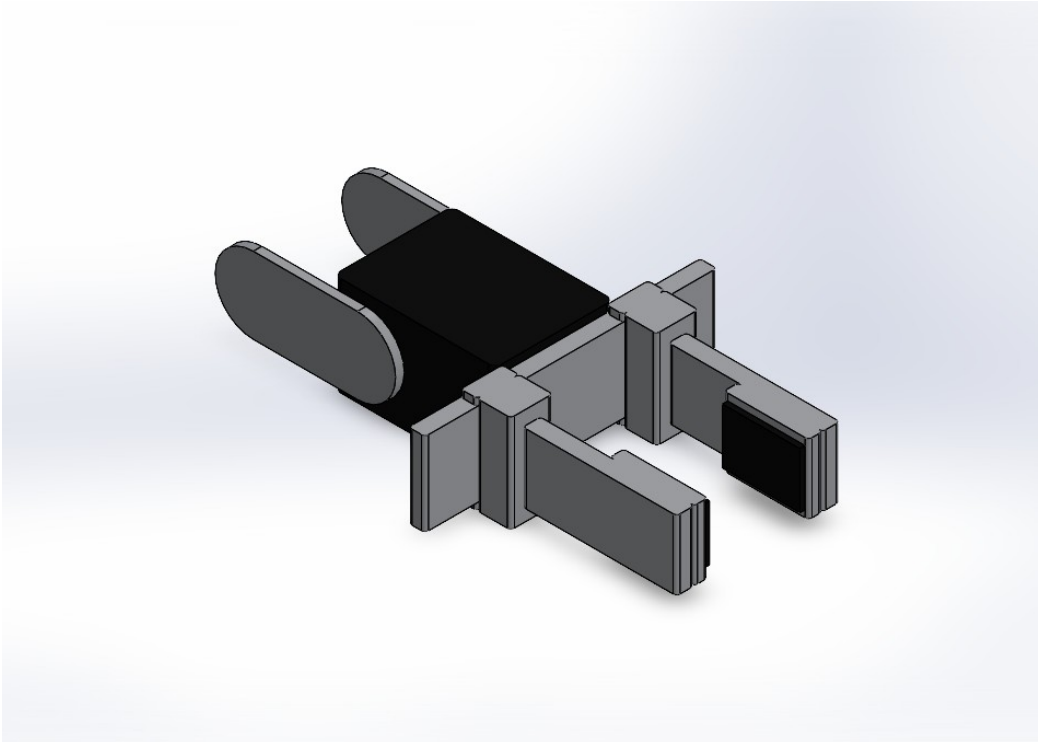
4. Planning pipeline used in the benchmark



- Collision checker: forward kinematics -> link capsules (r = 25 mm) vs obstacle spheres
- Metrics: success, time to first solution, joint-space path length, node count

4. Methodology: CAD and ROS 2 model

- 19 parametric parts: TB01-TB13 (TurtleBot3), OM01-OM06 (arm)
- Dimensions tied to Global Variables, e.g. J3_Z, J4_X, Hole_P, Tire_D
- Joints 1-4: real revolute mates; fingers: slider mates (6-64 mm opening)
- URDF generated from the same spec file, so CAD and URDF stay in step
- Same link / joint names as the official package



4. Kinematic chain

Joint	Parent -> child	Origin (m)	Axis	Limits
base_fixed	base_link -> link1	(-0.092, 0, 0.091)	-	fixed
joint1	link1 -> link2	(0.012, 0, 0.017)	z	+/-0.9 pi
joint2	link2 -> link3	(0, 0, 0.0595)	y	-0.57 pi ... 0.5 pi
joint3	link3 -> link4	(0.024, 0, 0.128)	y	-0.3 pi ... 0.44 pi
joint4	link4 -> link5	(0.124, 0, 0)	y	-0.57 pi ... 0.65 pi
gripper_left/right	link5 -> finger	(0.0817, +/-0.021, 0)	+/-y	-0.010 ... 0.019 m
end_effector_joint	link5 -> tool	(0.126, 0, 0)	-	fixed
wheel_left/right	base_link -> wheel	(0, +/-0.144, 0.023)	z	continuous

Same as turtlebot3_manipulation_description (ROS 2 Humble); checked automatically.

4. Equations (1/4): kinematics and free space

Forward kinematics

$$ {}^{0}T_{ee}(\mathbf{q}) = T_{base} \prod_{i=1}^{4} \left[\mathrm{Trans}(\mathbf{o}_i)\, \mathrm{Rot}(\hat{\mathbf{a}}_i,\, q_i) \right] \mathrm{Trans}(\mathbf{o}_{ee}) $$

Joint rotations

$$ \mathrm{Rot}(\hat{z}, q) = \begin{vmatrix} \cos q & -\sin q & 0 \\ \sin q & \cos q & 0 \\ 0 & 0 & 1 \end{vmatrix} \qquad \mathrm{Rot}(\hat{y}, q) = \begin{vmatrix} \cos q & 0 & \sin q \\ 0 & 1 & 0 \\ -\sin q & 0 & \cos q \end{vmatrix} $$

Free configuration space

$$ \mathcal{Q}_{free} = \{ \mathbf{q} :\ \mathbf{q}_{min} \le \mathbf{q} \le \mathbf{q}_{max},\quad d\left(\mathrm{link}_i(\mathbf{q}),\, c_j\right) > r_j + r_l \ \ \forall i, j,\quad z_i(\mathbf{q}) \ge z_{min} \} $$

4. Equations (2/4): collision, cost and RRT

Point-to-segment distance (collision test)

$$d(\overline{\mathbf{ab}}, \mathbf{c}) = \min_{t \in [0,1]} \| \mathbf{a} + t(\mathbf{b} - \mathbf{a}) - \mathbf{c} \|$$

Path length (cost)

$$L(\sigma) = \sum_{k=0}^{m-1} \| \mathbf{q}_{k+1} - \mathbf{q}_k \|_2$$

RRT: extend towards a random sample

$$\mathbf{q}_{new} = \mathbf{q}_{near} + \min\left(1, \frac{\eta}{\|\mathbf{q}_{rand} - \mathbf{q}_{near}\|}\right)(\mathbf{q}_{rand} - \mathbf{q}_{near})$$

RRT*: choose the cheapest parent

$$\mathbf{q}_{parent} = \arg \min_{\mathbf{q}_j \in \mathcal{Q}_{near}} [\, c(\mathbf{q}_j) + \|\mathbf{q}_j - \mathbf{q}_{new}\| \,]$$

4. Equations (3/4): RRT* and PRM

RRT*: rewire neighbours

$$c(\mathbf{q}_{new}) + \|\mathbf{q}_{new} - \mathbf{q}_j\| < c(\mathbf{q}_j) \;\Rightarrow\; \text{parent}(\mathbf{q}_j) \leftarrow \mathbf{q}_{new}$$

RRT*: neighbour radius

$$r_n = \min\left\{ \gamma \left(\frac{\ln n}{n}\right)^{1/d},\ \eta \right\}, \qquad \gamma > 2\left(1 + \frac{1}{d}\right)^{1/d} \left(\frac{\mu(\mathcal{Q}_{free})}{\zeta_d}\right)^{1/d}$$

RRT*: asymptotic optimality

$$P\left(\lim_{n\to\infty} c(\sigma_n) = c^*\right) = 1 \quad (\text{RRT}^*)$$

PRM: roadmap edges

$$E = \{(\mathbf{q}_i, \mathbf{q}_j) :\ \mathbf{q}_j \in k\text{NN}(\mathbf{q}_i),\ \overline{\mathbf{q}_i\mathbf{q}_j} \subset \mathcal{Q}_{free}\}, \qquad w_{ij} = \|\mathbf{q}_i - \mathbf{q}_j\|$$

4. Equations (4/4): PRM query and dynamics

PRM: Dijkstra query

$$D(v) = \min_{u:\,(u,v)\in E} [\, D(u) + w_{uv} \,], \qquad D(\mathbf{q}_s) = 0$$

Manipulator dynamics (tracking the planned path)

$$\mathbf{M}(\mathbf{q})\,\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\,\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}$$

Computed-torque control

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\left[\ddot{\mathbf{q}}_d + K_d\,\dot{\mathbf{e}} + K_p\,\mathbf{e}\right] + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}), \quad \mathbf{e} = \mathbf{q}_d - \mathbf{q}$$

Dynamics are shown for context; they are not simulated in this review.

4. The three algorithms

RRT

- 1. Start the tree T with q_s.
- 2. Sample q_rand in the joint limits (with probability 0.1 take q_g instead).
- 3. Find the nearest node q_near in T and step towards q_rand to get q_new (Eq. 6).
- 4. If the segment q_near -> q_new is collision-free, add q_new to T.
- 5. If q_new can reach q_g, stop and read the path back through the parents.

RRT*

- 1. Steps 1-3 as in RRT.
- 2. Collect the neighbours Q_near within radius r_n (Eq. 9).
- 3. Choose the parent that gives the lowest cost-to-come (Eq. 7).
- 4. Rewire: for each neighbour, if going through q_new is cheaper, make q_new its parent (Eq. 8).
- 5. Do not stop at the first solution; keep sampling until the time budget ends and keep the cheapest path.

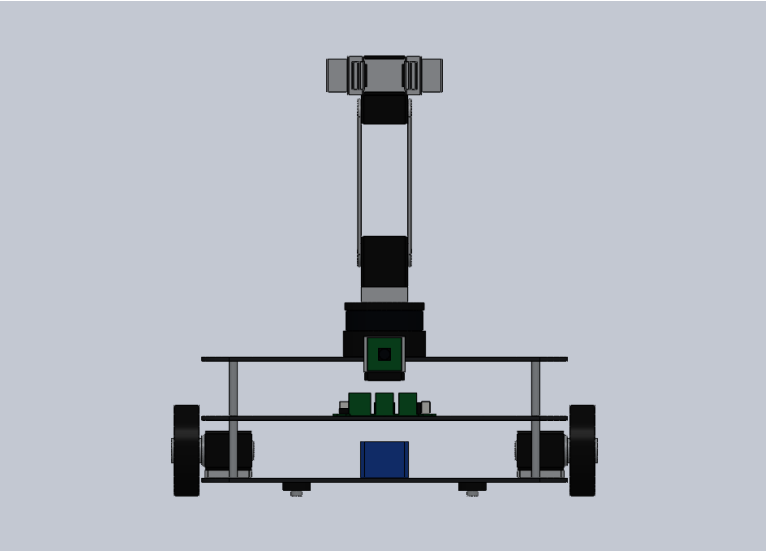
PRM

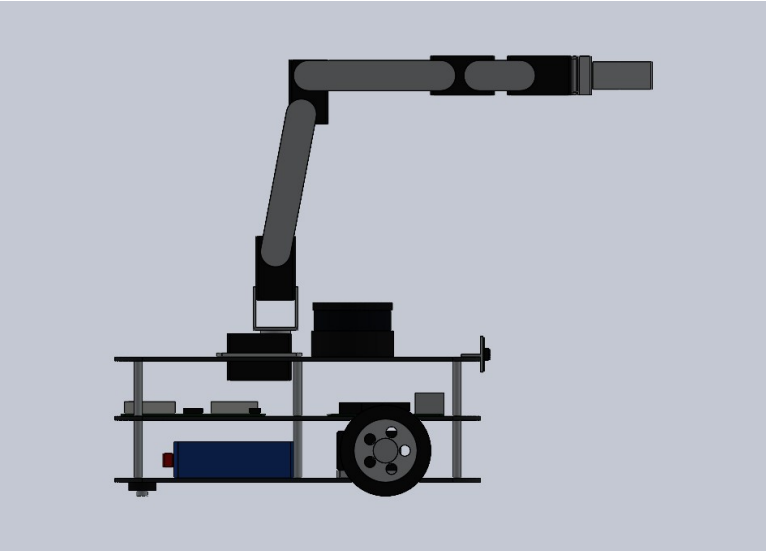
- 1. Learning phase: sample N collision-free configurations (plus q_s and q_g).
- 2. Connect each node to its k nearest neighbours if the straight segment is free (Eq. 11).
- 3. Query phase: run Dijkstra from q_s to q_g over the roadmap (Eq. 12).
- 4. If no path exists, add more nodes and repeat.

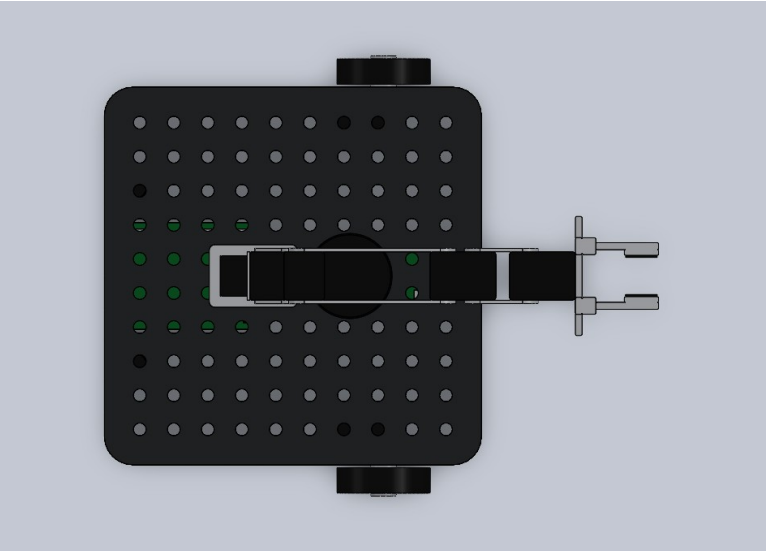
4. Benchmark setup

- Python implementation of FK, capsule collision check and the three planners
- 3 spherical obstacles; straight line start-to-goal is blocked (checked), so a detour is required
- Start (-1.2, -0.2, 0.6, 0.3) to goal (1.2, -0.2, 0.6, 0.3) rad; lower bound 2.40 rad
- RRT: step 0.4 rad, 10 % goal bias, stop at first solution
- RRT*: same step, rewiring, 10 s budget
- PRM: 1500 nodes, k = 10, Dijkstra query
- 10 runs per planner; metrics: success, time, path length, nodes

5. Results: CAD and ROS 2 package

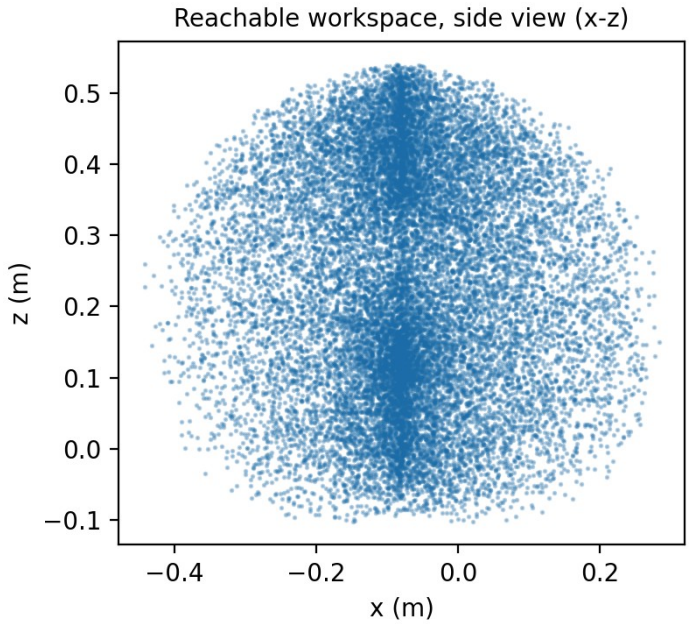


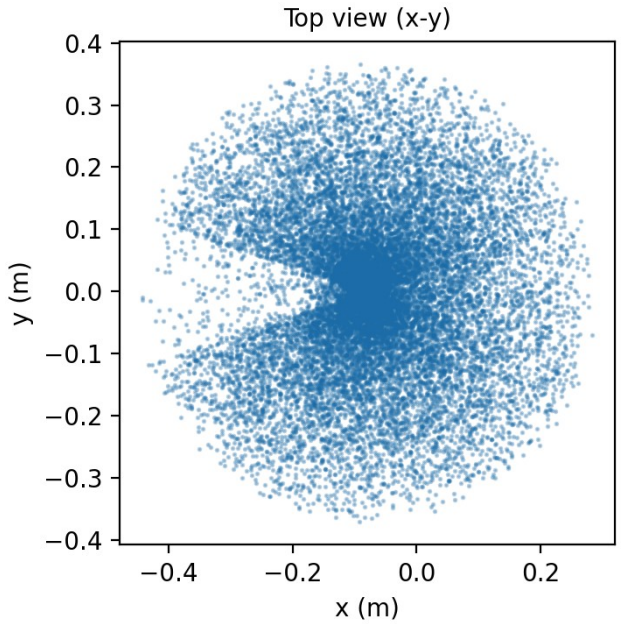




- 19 parts, 2 assemblies, 1 STEP file, 19 STL meshes
- Standalone URDF (20 links, 19 joints) and a drop-in xacro for Gazebo / ros2_control
- RViz launch with joint sliders; MoveIt 2 and Nav2 configs work unchanged

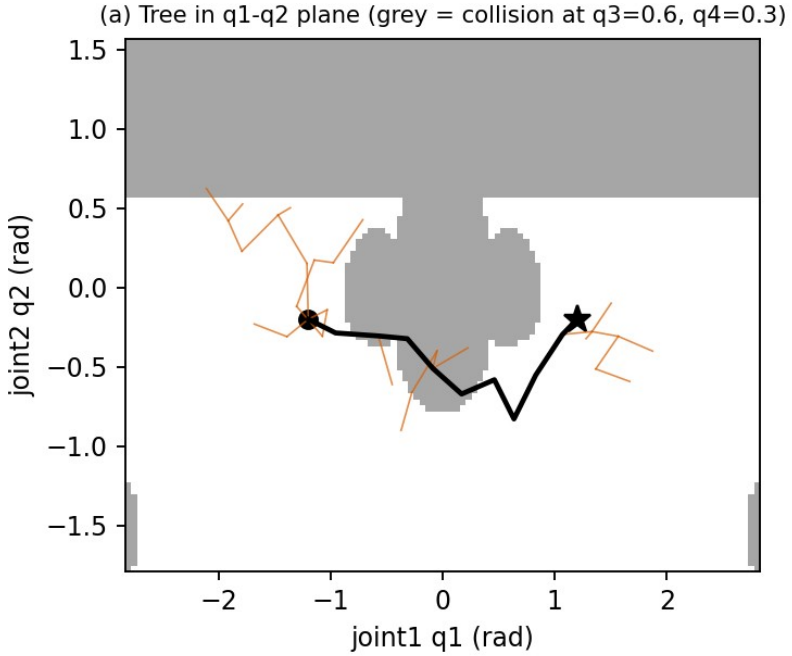
5. Results: arm workspace

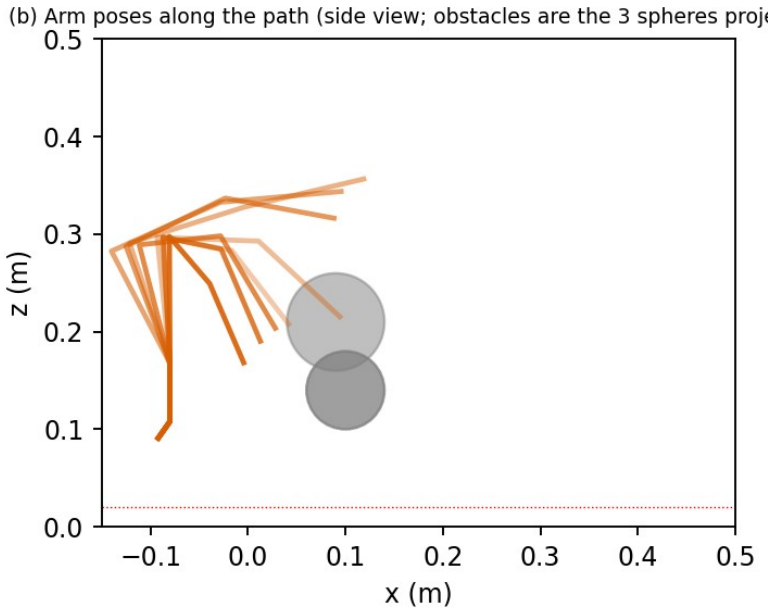


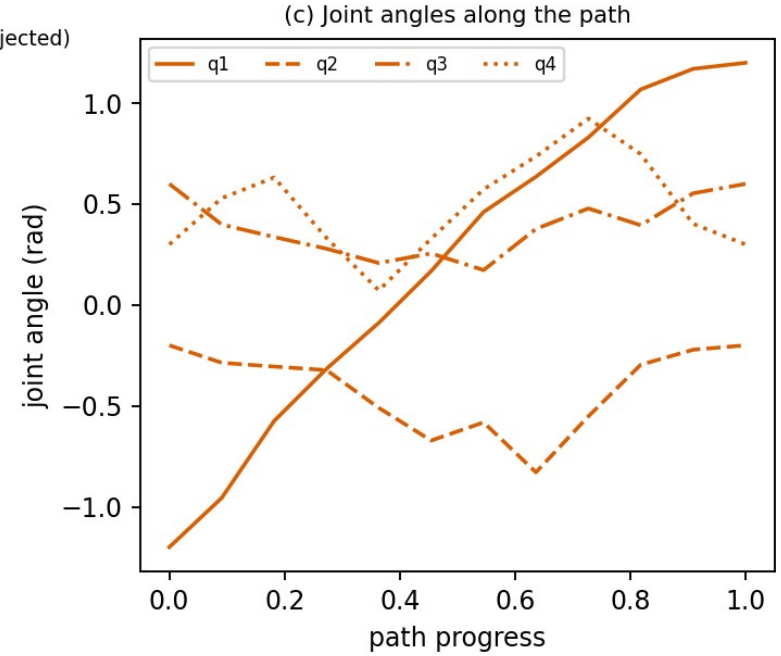


- Max reach from arm base (link1): 45.0 cm
- Max height above floor: 55.1 cm
- Max forward reach: 28.3 cm from base_link
- 20 000 sampled configurations

5. Simulation: RRT

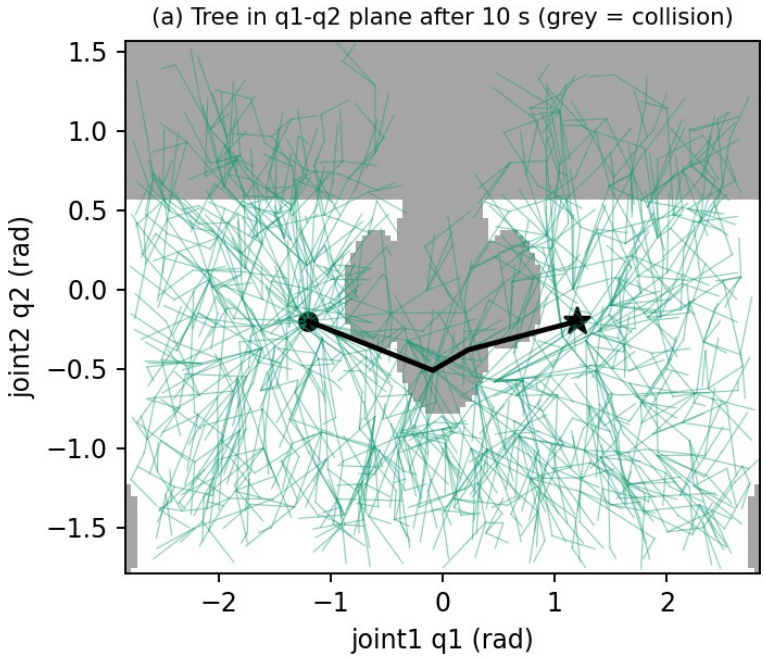


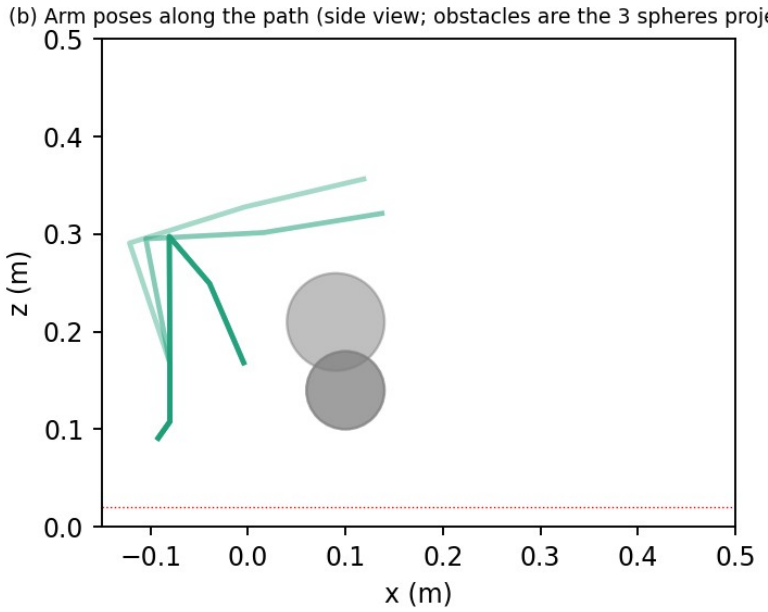


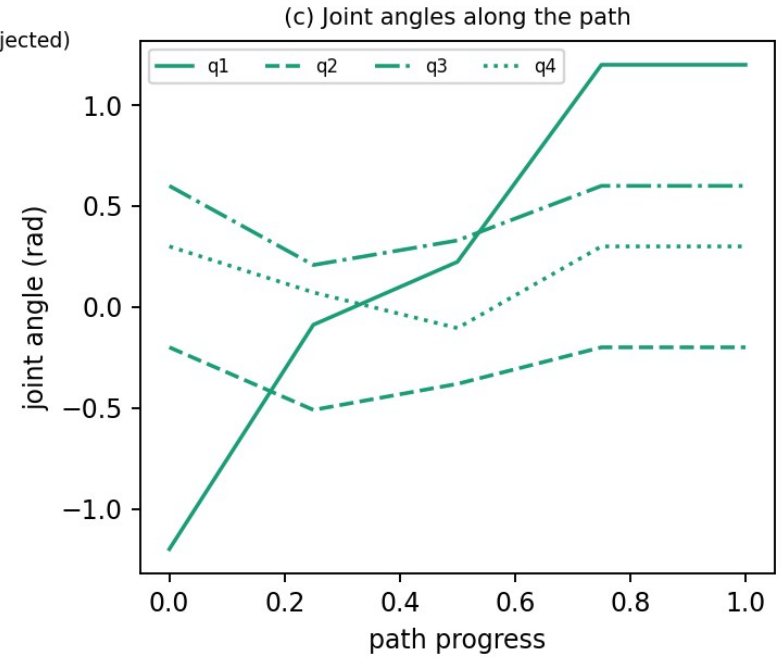


- Tree of only 39 nodes in this run (43 ms on average over 10 runs)
- Path 4.32 +/- 0.54 rad: 80 % above the lower bound, jagged
- Why: it extends towards the nearest node and stops at the first connection, so it is fast but never improves the path

5. Simulation: RRT*

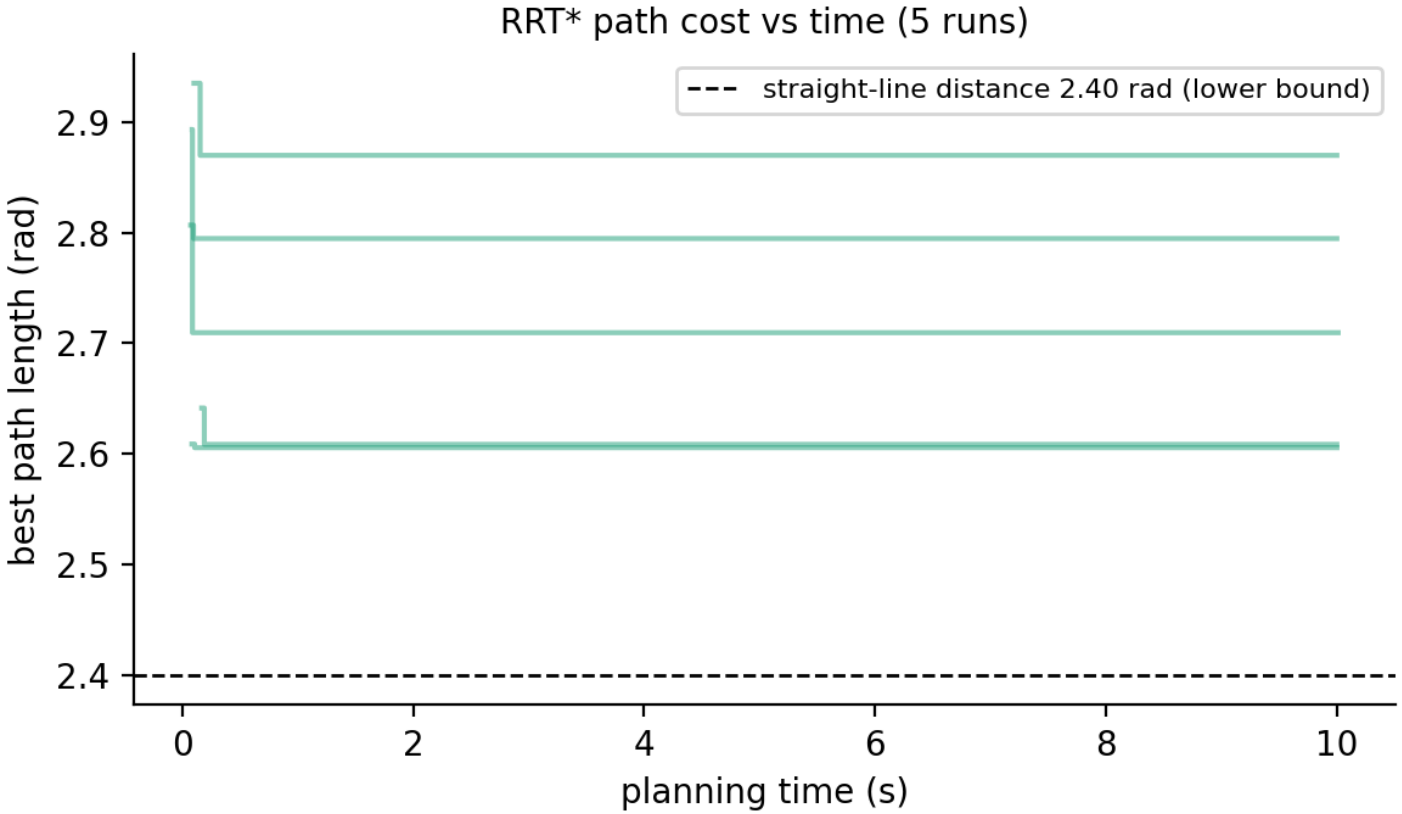






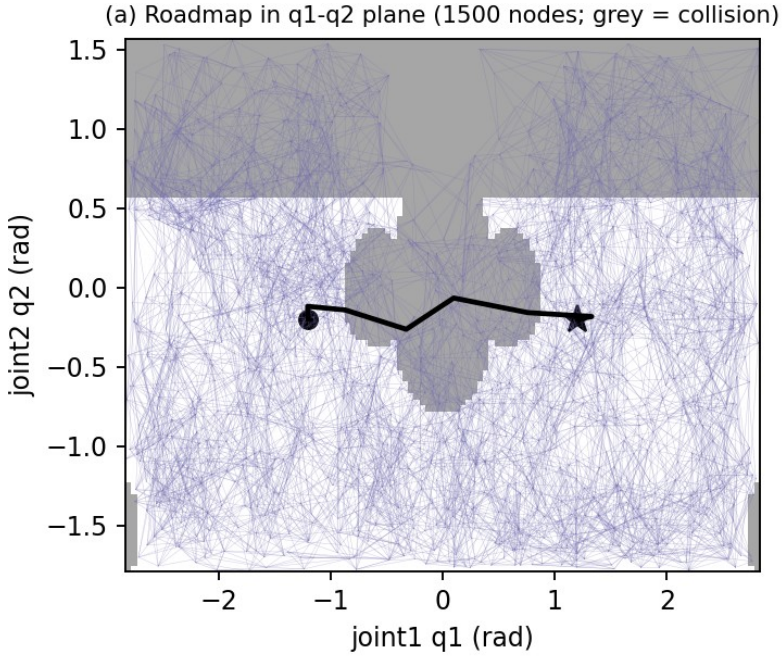
- About 1661 nodes after 10 s; first path after 87 ms
- Path 2.83 +/- 0.28 rad: only 18 % above the lower bound, smooth
- Why: best-parent choice and rewiring remove detours; cost is paid in computation (tree about 46x larger than RRT)

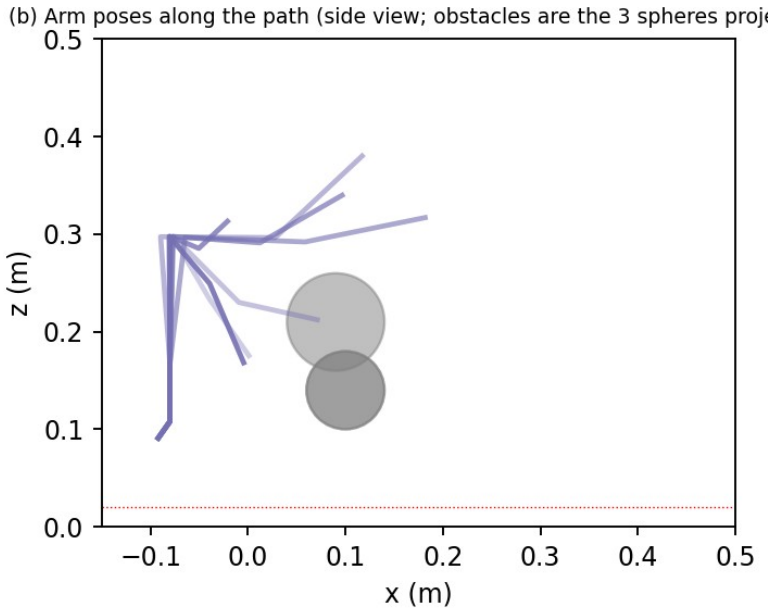
5. RRT* improves with time

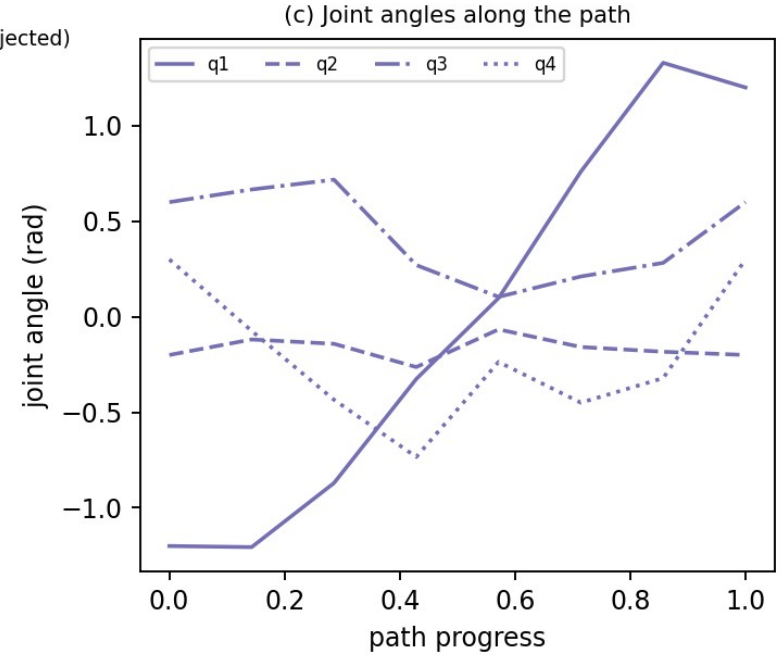


- Each curve starts at the first path and steps down as rewiring finds shorter routes
- The straight line (2.40 rad) is blocked, so the true optimum is above it
- Can be stopped at any time and still returns a valid path

5. Simulation: PRM







- 1500 nodes, 7417 edges in this run; 10.7 s to build, then a graph search
- Path 3.64 +/- 0.27 rad: limited to roadmap edges
- Why: effort is spent up front; later queries in the same static scene are almost free

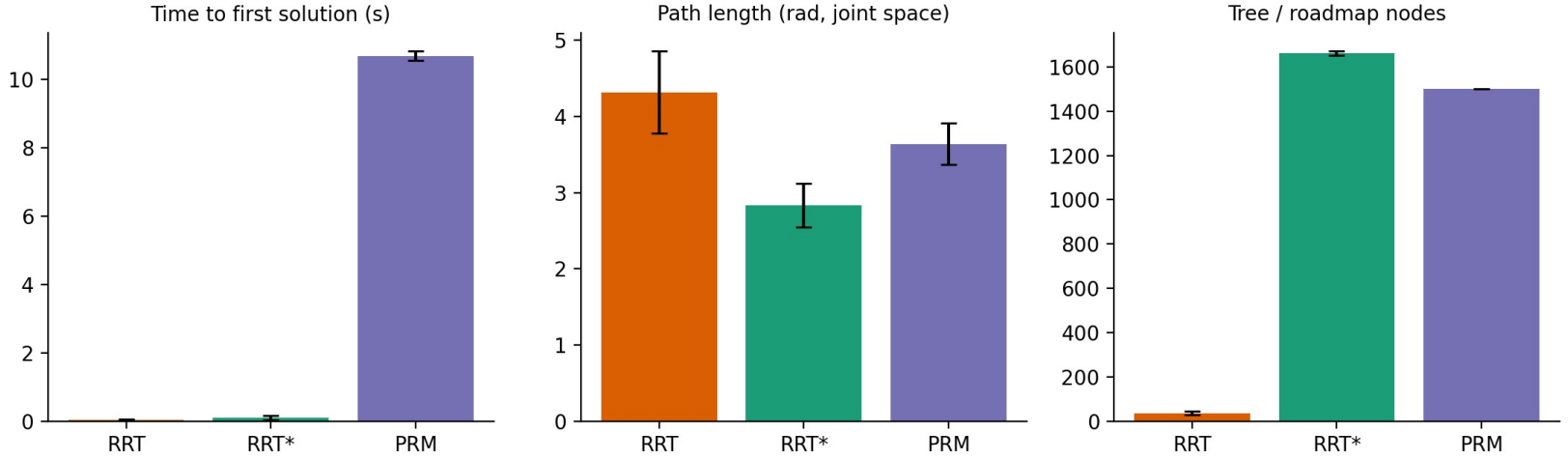
5. PRM: effect of roadmap size

Nodes N	Success	Build + query (s)	Mean path length (rad)
100	10/10	1.4	4.13
250	10/10	2.7	4.22
500	10/10	4.5	3.81
1000	10/10	7.5	3.50
1500	10/10	10.7	3.64

- Build time rises steadily with N
- Path length tends to fall as N grows, but neighbouring sizes differ by less than the run-to-run spread (about 0.27 rad)
- All sizes solved the problem 10/10

5. Results: comparison

Algorithm	Success	First solution (ms)	Path length (rad)	Above lower bound	Nodes
RRT	10/10	43	4.32	80 %	36
RRT*	10/10	87	2.83	18 %	1661
PRM	10/10	10694	3.64	52 %	1500



- RRT: fastest
- RRT*: shortest, most repeatable
- PRM: slow build, reusable

Our own Python implementation (not OMPL); mean of 10 runs. PRM time = roadmap build + query.

5. Justification: which algorithm when?

Algorithm	Best used when	Trade-off
RRT	One-off motion, need an answer fast	Paths long and jagged; no improvement
RRT*	One-off motion, path quality matters; default for this robot	Uses full time budget; larger tree
PRM	Repeated moves in a fixed scene (workstation)	Slow first query; rebuild if obstacles move

- Mobile manipulator arrives at new places, so the scene changes: RRT / RRT* fit best
- The MoveIt 2 study will test whether RRT* stays the best default

5. Limitations

- Simple scene (3 spheres); fixed base; no self-collision, gripper fingers or mesh collision
- Our own Python planners, not OMPL: times not comparable with MoveIt
- 10 runs per planner: rough picture of the spread
- Dynamics shown as equations only, not simulated
- So: repeat in MoveIt 2 + OMPL + Gazebo

6. Conclusion and future work

- Delivered: parametric CAD + ROS 2 model whose kinematics match the official robot
- Simulated and justified RRT, RRT* and PRM on it: RRT quickest, RRT* best paths, PRM reusable
- Next: MoveIt 2 + OMPL + Gazebo (RRTkConfigDefault, RRTstarkConfigDefault, PRMkConfigDefault)
- Next: RRT-Connect, Informed RRT*, arm dynamics with computed-torque control, base + arm planning, real-robot pick-and-place

7. References

[1] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Tech. Rep. TR 98-11, Computer Science Dept., Iowa State University, 1998.

[2] L. E. Kavraki, P. Svestka, J.-C. Latombe and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," IEEE Trans. Robotics and Automation, vol. 12, no. 4, pp. 566-580, 1996.

[3] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," Int. J. Robotics Research, vol. 30, no. 7, pp. 846-894, 2011.

[4] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," Proc. IEEE ICRA, pp. 995-1001, 2000.

[5] I. A. Sucan, M. Moll and L. E. Kavraki, "The Open Motion Planning Library," IEEE Robotics & Automation Magazine, vol. 19, no. 4, pp. 72-82, 2012.

[6] S. Chitta, I. Sucan and S. Cousins, "MoveIt!," IEEE Robotics & Automation Magazine, vol. 19, no. 1, pp. 18-19, 2012.

[7] S. Macenski, T. Foote, B. Gerkey, C. Lalancette and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, vol. 7, no. 66, 2022.

[8] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," Proc. IEEE/RSJ IROS, pp. 2149-2154, 2004.

[9] S. Macenski, F. Martin, R. White and J. G. Clavero, "The Marathon 2: A Navigation System," Proc. IEEE/RSJ IROS, 2020.

[10] B. Siciliano, L. Sciavicco, L. Villani and G. Oriolo, Robotics: Modelling, Planning and Control. Springer, 2009.

[11] ROBOTIS Co., Ltd., "TurtleBot3 and OpenMANIPULATOR-X e-Manual," https://emanual.robotis.com (accessed 2026).

[12] ROBOTIS-GIT, "turtlebot3_manipulation" (ROS 2 Humble branch), https://github.com/ROBOTIS-GIT/turtlebot3_manipulation.

[13] J. D. Gammell, S. S. Srinivasa and T. D. Barfoot, "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," Proc. IEEE/RSJ IROS, pp. 2997-3004, 2014.

Thank you

Thank you | Questions?